

ORM Lab (CodeGrade)

<https://github.com/learn-co-curriculum/python-p3-v2-orm-lab> <https://github.com/learn-co-curriculum/python-p3-v2-orm-lab/issues/new>

Learning Goals

- Map a Python class to a database table.
- Persist a one-to-many relationship.
- Define property methods to manage attributes.

Instructions

This is a **test-driven lab**. Run `pipenv install` to create your virtual environment and `pipenv shell` to enter the virtual environment. Then run `pytest -x` to run your tests. Use these instructions and `pytest` 's error messages to complete your work in the `lib/` folder.



This lab involves enhancing the company data model from the ORM lessons. You will implement a class named `Review` that encapsulates an annual performance review for an employee. There is a one-to-many relationship between `Employee` and `Review`.

- An employee may have many reviews.
- A review is for one employee.
- A review includes an attribute for the `year` as well as a `summary` of the employee's performance.

The `Department` and `Employee` classes contain the completed solutions from the ORM lessons. You will add functionality for the new `Review` class in this lab.

Environment

The environment is initialized in `lib/__init__.py` to generate a `sqlite3.Connection` object `CONN`, and a `sqlite3.Cursor` object `CURSOR`.

Implementing `Review`

The `Review` class contains the following methods already implemented for you:

- `__init__` initializes attributes for the review `year`, `summary`, and associated `employee`. It also initializes the `id` attribute to a default value of `None`.
- `__repr__` returns a formatted string containing the attribute values.
- `create_table` creates a "reviews" database table.
- `drop_table` drops the "reviews" database table.

Edit the `Review` class to add the following ORM methods (you will add property methods in a subsequent step):

`save()`

The instance method `save()` should persist the `Review` object to the "reviews" table:

- Insert a new row with the `year`, `summary`, and `employee_id` values of the current `Review` instance.
- Update the object `id` attribute using the primary key value of new row.
- Save the object in local dictionary using table row's PK as dictionary key.

`create()`

This is a class method that should:

- Create a new `Review` instance using the parameter values.
- Save the new `Review` instance to the "reviews" table.
- Return the new `Review` instance.

Think about how you can re-use the `save()` method to help with this one.

`instance_from_db()`

This class method should return a `Review` instance having the attribute values from the table row. You should check the dictionary for an existing instance using the row's primary key, and set the instance attributes to the row data if found. If the dictionary does not contain a previously persisted object with that id, create a new `Review` instance from the row data and add it to the dictionary. The method should return the cached object.

`find_by_id()`

This class method takes in an `id` as a parameter, and should return a single `Review` instance corresponding to the row in the "reviews" table with that same id, or `None` if no such row exists in the table.

`update()`

This instance method should update the `year`, `summary` and `employee_id` columns for a "reviews" table row based on the `id` of the current object.

`delete()`

This instance method should delete a "reviews" table row based on the `id` of the current object. It will also remove the instance from the `all` dictionary and set the current object's `id` attribute to `None`.

`get_all()`

This class method should return a list of `Review` instances for every row in the "reviews" table.

You can test your methods by running the tests in the "lib/testing/review_orm_test.py" file:

```
pytest lib/testing/review_orm_test.py
```

You can also experiment by running `python lib/debug.py` and calling the new ORM methods at the `ipdb>` prompt:

```
ipdb> Review.get_all()
```

Review Properties

Let's add property methods to set rules for the `Review` attributes as follows:

- `year` should be an integer that is greater than or equal to 2000.
- `summary` should be a non-empty string.
- `employee_id` should be the id of an `Employee` class instance that has been persisted into the "employees" table.

You can test your methods by running the tests in the "lib/testing/review_property_test.py" file:

```
pytest lib/testing/review_property_test.py
```

Update `Employee` to get a list of associated `Review` instances

Update the `Employee` class with a new method `reviews()` for getting associated `Review` instances that have been persisted to the database.

`reviews()`

This instance method should query the "reviews" table to get all rows where the foreign key column `employee_id` matches the id of the current `Employee` instance. The method should return a list of `Review` instances for each matching table row.

NOTE: To avoid issues with circular imports, embed import the `Review` class within the `reviews()` method rather than at the module level.

You can test your methods by running the tests in the "lib/testing/employee_orm_test.py" file:

```
pytest lib/testing/employee_orm_test.py
```

Submit your completed lab

Check to make sure all tests pass:

```
pytest -x
```

Once all of your tests pass, commit and push your work using `git` to submit your solution.

This tool needs to be loaded in a new browser window

Load ORM Lab (CodeGrade) in a new window